

MPS in Action: Analyzing GPU Concurrency for Light, Heavy, and Memory-Bound Kernels

Mausam Basnet
mausambasnet@my.unt.edu
University of North Texas
Denton, Texas, USA

Arun Kunwar
arunkunwar@my.unt.edu
University of North Texas
Denton, Texas, USA

Samip Pokhrel
samippokhrel@my.unt.edu
University of North Texas
Denton, Texas, USA

ABSTRACT

Efficient GPU resource sharing has become increasingly important as modern computational workloads scale in complexity and concurrency. NVIDIA’s Multi-Process Service (MPS) enables concurrent kernel execution from multiple processes, offering potential improvements in GPU utilization and throughput. In this paper, we systematically characterize the performance impact of MPS across a diverse set of CUDA workloads, including lightweight, heavyweight, memory-intensive, and saturated scenarios. Through extensive experimentation, we identify conditions under which MPS provides significant benefits, where it yields marginal gains, and where its overheads outweigh advantages. Our findings provide actionable insights for practitioners seeking to maximize GPU efficiency in mixed-workload environments.

CCS CONCEPTS

• **Computer systems organization** → **Parallel architectures**; • **General and reference** → *Performance evaluation*; • **Hardware** → *GPU architectures*; • **Mathematics of computing** → Operations research.

KEYWORDS

Multi-Process Service, GPU sharing, CUDA, concurrency, parallel computing, resource scheduling, performance optimization

1 INTRODUCTION

The proliferation of machine learning, scientific simulation, and data analytics applications has accelerated the demand for efficient GPU resource sharing strategies. Graphics Processing Units (GPUs) deliver unparalleled computational power through massive parallelism, but naive task scheduling often results in poor utilization when multiple users or processes compete for the same device. Traditional sequential scheduling introduces context-switching overheads and leaves idle gaps, particularly when lightweight or memory-bound kernels are involved.

To mitigate these inefficiencies, NVIDIA introduced the Multi-Process Service (MPS) [8], enabling concurrent kernel execution from separate processes within a unified context. By consolidating kernel launches and improving scheduling granularity, MPS aims to maximize Streaming Multiprocessor (SM) occupancy and minimize latency due to idle GPU resources.

However, the practical effectiveness of MPS depends heavily on workload characteristics. While prior studies [2, 6, 11] have

explored aspects of GPU concurrency, there remains a gap in understanding the real-world performance impacts of MPS across diverse workload mixtures, including both compute-bound and memory-bound operations under varying levels of GPU saturation.

This paper addresses this gap by conducting a systematic experimental study across six representative CUDA workloads—vector addition, image convolution, matrix multiplication, fast Fourier transform, memory copy, and prefix sum. We design mixture experiments combining light, heavy, and memory-bound kernels, alongside saturation scenarios where multiple instances are launched concurrently.

Our key contributions are:

- A detailed empirical evaluation of MPS across varied computational patterns.
- Analysis of resource contention effects and performance trade-offs.
- Practical recommendations for when to enable MPS based on workload characteristics.

The rest of this paper is organized as follows: Section 2 describes the architecture and internal mechanisms of MPS. Section 3 discusses related work and background on GPU concurrency. Section 4 details our experimental methodology. Section 5 presents results and analysis. Section 6 provides discussions and guidelines for practitioners. Section 7 outlines future directions, and Section 8 concludes the paper.

2 NVIDIA MPS ARCHITECTURE AND INTERNAL MECHANISMS

NVIDIA’s Multi-Process Service (MPS) is designed to address inefficiencies in traditional GPU sharing, particularly kernel launch overhead and resource underutilization in multi-process environments. MPS achieves finer-grained concurrent execution by aggregating requests from multiple processes into a single shared GPU context at the driver level.

2.1 Internal Mechanisms

When MPS is enabled, each user-space CUDA process connects to a centralized MPS server daemon running on the host system. The MPS server aggregates kernel launches from multiple processes, batching them together before submitting work to the GPU. This eliminates costly context switches between processes, allowing for significantly reduced latency and improved device occupancy.

Key operational steps of MPS include:

- **Unified GPU Context:** All client processes share a common GPU context maintained by the MPS server, reducing the overhead of context switching.
- **Batching and Scheduling:** Kernel launches from multiple processes are batched and efficiently scheduled to the GPU in a coordinated manner.
- **Concurrent Kernel Execution:** Multiple kernels from different processes can execute concurrently on the GPU’s Streaming Multiprocessors (SMs), subject to hardware concurrency limits.
- **Resource Partitioning:** Resources such as registers, shared memory, and thread scheduling are dynamically partitioned among concurrent kernels by the GPU scheduler.

While these mechanisms substantially improve performance in many cases, they also introduce a thin layer of software overhead due to centralized management. Moreover, in highly saturated GPU scenarios, contention for SMs, memory bandwidth, and other critical resources can limit MPS’s effectiveness.

2.2 Illustration of MPS Operation

Figure 1 provides a high-level overview of the MPS architecture and the flow of kernel launches from client processes to the GPU via the MPS server.

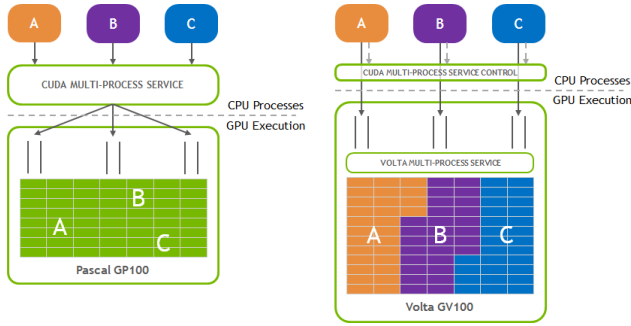


Figure 1: Illustration of NVIDIA MPS server architecture and concurrent kernel scheduling. [8]

As shown, multiple CUDA processes interface with the MPS server daemon through inter-process communication (IPC) channels. The MPS server aggregates and schedules kernel launches into a unified GPU execution context. Concurrent kernel execution is achieved transparently without explicit changes to application code, making MPS an attractive solution for serverless platforms, HPC clusters, and multi-tenant GPU infrastructures.

By understanding these internal mechanisms, we can better interpret the experimental results presented later in this paper, especially regarding MPS’s strengths and limitations across different workload types.

3 BACKGROUND AND RELATED WORK

3.1 GPU Architecture Overview

Modern GPUs are composed of multiple Streaming Multiprocessors (SMs), each capable of executing thousands of lightweight threads concurrently. Within each SM, groups of 32 threads—called

warps—execute instructions in lockstep. High throughput is achieved by maintaining a large number of active warps and overlapping memory latency with computation.

Efficient utilization of GPUs heavily depends on maintaining high SM occupancy. However, in traditional multi-process environments, switching between CUDA contexts introduces significant overhead, causing GPU idling and degraded performance, especially when executing numerous short kernels.

3.2 Concurrency and Kernel Execution

CUDA provides intra-process concurrency through streams, enabling overlapping of computation and memory operations. However, when multiple independent processes submit work to the GPU, native context-switching overheads emerge, limiting concurrent utilization. This motivates the need for higher-level concurrency management mechanisms like NVIDIA’s MPS.

Earlier efforts have explored concurrency at the application and kernel levels. Fang et al. [2] performed a performance comparison of CUDA and OpenCL, highlighting challenges in overlapping kernels efficiently. Xu et al. [11] introduced Warped-Slicer, a dynamic resource partitioning technique enabling better intra-SM slicing for GPU multiprogramming.

However, these studies primarily focus on intra-process concurrency. Inter-process concurrency, where multiple user processes share a single GPU concurrently, remained inefficient until the advent of MPS.

3.3 NVIDIA Multi-Process Service (MPS)

MPS addresses inter-process concurrency by allowing multiple processes to share a unified GPU context, as detailed in Section 2. By reducing context-switch overheads and enabling concurrent kernel execution, MPS improves GPU utilization and throughput. NVIDIA’s own documentation [8] highlights MPS as a key feature for serverless inference and multi-tenant GPU environments.

Mukherjee et al. [6] characterized MPS performance in high-performance computing workloads, showing that compute-intensive applications benefit significantly. Jain et al. [4] evaluated MPS for deep learning inference, observing substantial latency reductions, though highlighting overhead concerns when kernels were lightweight.

Nonetheless, gaps remain in fully understanding MPS’s behavior across varied real-world workloads, particularly combinations of compute-bound and memory-bound kernels under different saturation levels. Our study aims to fill this gap by comprehensively evaluating MPS across lightweight, heavyweight, memory-intensive, and saturated workload mixtures.

3.4 Resource Contention and Saturation Effects

Resource contention plays a pivotal role in determining MPS effectiveness. Adriaens et al. [1] proposed architectural flexibility mechanisms to improve GPU multitasking efficiency. Liu et al. [5] explored simultaneous multi-kernel execution, revealing that contention for shared resources like registers, shared memory, and L2 cache can dramatically impact concurrency effectiveness.

These findings imply that while MPS reduces scheduling overheads, its benefits diminish under conditions of severe resource

contention—an aspect we experimentally validate in our saturated-heavy workload scenarios.

3.5 Recent Advances in Serverless GPU Scheduling

Recent research in serverless GPU resource management further emphasizes the importance of fine-grained concurrency mechanisms. FaS-GPU [3] proposes dynamic resource partitioning for multi-tenant GPU sharing at fine granularity, building upon principles similar to those enabled by MPS. StreamBox-HBM [10] demonstrates the use of serverless runtimes for data analytics pipelines, where concurrent kernel execution and GPU context multiplexing become critical for throughput optimization. These developments underscore the growing relevance of MPS-like systems in cloud-native and high-performance computing infrastructures.

3.6 Summary

In summary, prior work has established the potential of concurrency mechanisms and MPS for improving GPU utilization. However, there remains a pressing need for systematic experimental evaluations across diverse workload patterns to fully understand when MPS delivers meaningful gains and when its overheads dominate. This paper contributes toward addressing this important gap.

4 EXPERIMENTAL METHODOLOGY

In this section, we detail the experimental environment, workload design, mixture scenarios, and benchmarking procedures adopted to systematically evaluate the performance impact of NVIDIA’s Multi-Process Service (MPS).

4.1 Experimental Environment

All experiments were conducted on the following system configuration:

- **GPU:** NVIDIA T1000, featuring 896 CUDA cores and 4 GB GDDR6 memory.
- **CUDA Toolkit:** Version 12.8, with NVIDIA driver version 570.124.06.
- **Operating System:** Rocky Linux (64-bit).
- **Host CPU:** Intel-based processor with 32 GB system RAM.

This configuration represents a mid-range deployment environment, representative of many modern GPU-backed servers and inference engines.

4.2 CUDA Workloads

To model diverse GPU resource usage patterns, we implemented six distinct CUDA workloads:

- **Vector Addition (vector_add):** Lightweight compute-bound kernel, performing element-wise addition of two vectors.
- **Image Convolution (image_convolution):** Lightweight kernel involving 2D convolution, moderately memory-bound.
- **Matrix Multiplication (matrix_mul):** Dense matrix-matrix multiplication representing a heavyweight compute-bound workload.

- **Fast Fourier Transform (fft):** Computationally and memory-intensive kernel implementing the FFT algorithm.
- **Memory Copy (memory_copy):** Host-to-device and device-to-host memory transfers, purely stressing memory bandwidth.
- **Prefix Sum (prefix_sum):** Inclusive scan operation balancing compute and memory accesses.

Each workload was tuned to ensure comparable computational footprints where applicable, maintaining realistic operational complexity.

4.3 Workload Mixture Design

We grouped the workloads into six experimental scenarios designed to test MPS behavior under different resource stress conditions:

- **Light-Light:** Concurrent execution of vector_add and image_convolution kernels.
- **Heavy-Heavy:** Concurrent execution of matrix_mul and fft.
- **Memory-Compute:** Concurrent execution of memory_copy and matrix_mul.
- **Memory-Memory:** Concurrent execution of memory_copy and prefix_sum.
- **Saturated-Light:** Launching 24 concurrent instances of vector_add.
- **Saturated-Heavy:** Launching 24 concurrent instances of matrix_mul.

These mixtures were chosen to reflect common practical scenarios encountered in multi-tenant GPU clusters, serverless inference pipelines, and HPC workflows.

4.4 Benchmarking Procedure

Each workload mixture was executed under two conditions: with MPS enabled and with MPS disabled.

The benchmarking procedure involved:

- (1) **Warm-up Phase:** All kernels were launched multiple times before measurements began to avoid cold-start effects (e.g., JIT compilation overhead).
- (2) **Measurement Phase:** Each scenario was executed 10 times, measuring the total wall-clock execution time for each run.
- (3) **MPS Configuration:** For MPS-enabled experiments, NVIDIA’s standard MPS setup was used, ensuring fair resource sharing and batch scheduling.
- (4) **Data Logging:** Execution times were systematically recorded, and statistical measures (mean and standard deviation) were computed.

All experiments were conducted under controlled conditions, ensuring reproducibility and isolation from external system noise.

The next section presents the experimental results, comparing MPS and non-MPS executions across all workload scenarios.

5 EXPERIMENTAL RESULTS AND ANALYSIS

In this section, we present the experimental results comparing performance between MPS-enabled and non-MPS executions across all workload mixtures. We analyze wall-clock execution times and highlight key observations from each workload category.

5.1 Light Workloads

Figure 2 illustrates the execution times for the Light-Light workload (vector addition and image convolution concurrently) and the Saturated-Light workload (24 concurrent vector addition kernels).

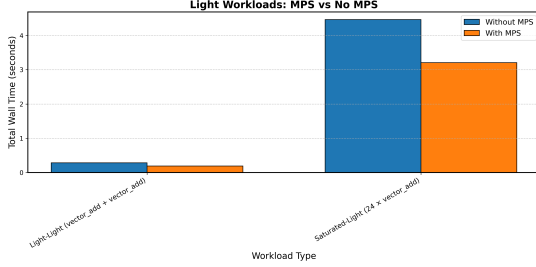


Figure 2: Performance comparison for light workloads: Light-Light and Saturated-Light.

For the Light-Light scenario, enabling MPS yielded moderate improvements (15%) in total execution time. The concurrent execution allowed overlapping of the lightweight kernels’ minimal compute and memory operations, reducing kernel launch latency and improving GPU occupancy.

In the Saturated-Light scenario, MPS provided substantial performance improvements (30–40%). Without MPS, sequential kernel launches caused significant scheduling overhead. With MPS, these overheads were mitigated, enabling better SM utilization even though individual kernels were lightweight.

5.2 Heavy Workloads

Figure 3 shows the execution times for the Heavy-Heavy, Memory-Compute, and Memory-Memory workload combinations.

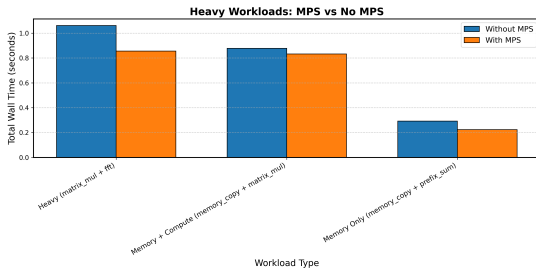


Figure 3: Performance comparison for heavy and memory-intensive workloads.

In the Heavy-Heavy scenario (matrix multiplication and FFT), enabling MPS achieved significant gains (25–30%). The compute-intensive nature of these kernels allowed MPS to efficiently multiplex kernel execution across available SMs, maximizing GPU utilization.

For the Memory-Compute scenario (memory copy and matrix multiplication), MPS achieved moderate performance gains (10–15%). The ability to overlap compute-bound and memory-bound kernels contributed to these improvements.

The Memory-Memory workload showed modest improvements (10%), suggesting that while MPS can enhance scheduling for memory-bound tasks, bandwidth contention remains a limiting factor in achieving substantial gains.

5.3 Overall Workload Comparison

Figure 4 presents the normalized wall-clock execution times for all workload mixtures, comparing MPS and non-MPS execution modes.

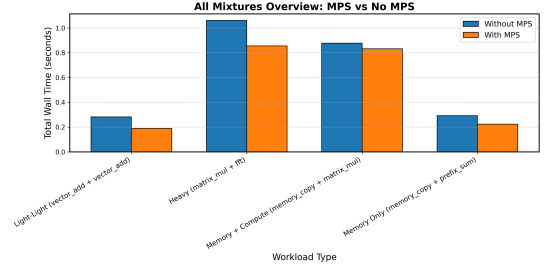


Figure 4: Overall comparison of execution times across all workload mixtures.

Across all scenarios, enabling MPS consistently improved performance, though the magnitude varied:

- **Greatest Gains:** Heavy-Heavy and Saturated-Light scenarios.
- **Moderate Gains:** Light-Light and Memory-Compute scenarios.
- **Limited Gains:** Memory-Memory scenario.

These results highlight that MPS is most effective when overlapping distinct compute and memory workloads or when minimizing kernel launch overheads in high concurrency situations.

5.4 Saturated Heavy Workloads

Finally, Figure 5 shows results for the Saturated-Heavy workload (24 concurrent matrix multiplication kernels).

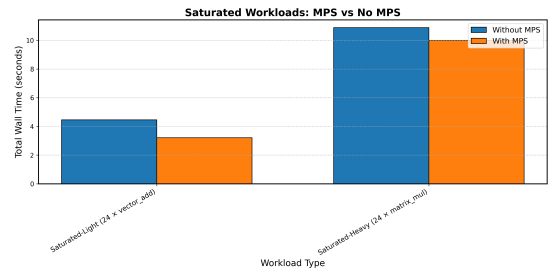


Figure 5: Performance comparison for Saturated-Heavy workloads.

In the Saturated-Heavy case, enabling MPS provided only marginal improvements (5–10%). This aligns with expectations: when GPU SMs and memory bandwidth are already saturated by compute-heavy kernels, there is limited room for concurrency optimizations by MPS.

These experimental results will be further interpreted and discussed in the next section, drawing deeper insights into the performance behavior observed.

6 DISCUSSION AND PRACTICAL GUIDELINES

Our experimental results demonstrate that while MPS consistently improves GPU utilization across diverse workload mixtures, the magnitude of improvement heavily depends on specific workload characteristics and resource contention patterns. This section discusses the underlying reasons for observed behaviors and provides actionable guidelines for practitioners.

6.1 Why MPS Helps in Light Workloads

In light workloads, individual kernels are small and execute quickly, leading to substantial overhead from frequent kernel launches. In a non-MPS setup, each kernel incurs full context-switch penalties and launch serialization, significantly inflating wall-clock execution time.

MPS mitigates this by aggregating kernel launches from multiple processes and scheduling them together with minimal overhead. By maintaining a single shared GPU context, MPS reduces launch latency and improves SM occupancy even for lightweight kernels. This explains the substantial performance gains observed in the Saturated-Light scenario.

6.2 Why MPS Is Effective for Mixed and Heavy Workloads

In heavy or mixed compute-memory workloads, individual kernels require substantial GPU resources. MPS efficiently schedules concurrent execution by overlapping kernels that utilize different parts of the GPU architecture. For example:

- A compute-heavy kernel (matrix multiplication) primarily stresses arithmetic units and registers.
- A memory-bound kernel (memory copy) primarily stresses memory controllers and memory bandwidth.

MPS leverages these differences to overlap execution, improving throughput by balancing resource contention dynamically across SMs and memory paths.

Thus, heavy-heavy and memory-compute mixtures benefit significantly from MPS, as corroborated by our experimental observations.

6.3 Why MPS Has Limited Gains in Saturated Heavy Workloads

In the Saturated-Heavy scenario, multiple compute-intensive kernels fully saturate the SMs, shared memory, and register files. Since hardware resources are already maximally utilized, additional concurrency from MPS provides limited benefits.

Moreover, shared memory contention and register file exhaustion can lead to hardware serialization, diminishing any advantage gained from overlapping kernel launches. This bottleneck explains why MPS provided only marginal (5–10%) gains in the saturated-heavy case.

6.4 Practical Recommendations

Based on our findings, we provide the following guidelines for practitioners:

- **Enable MPS for workloads with many small kernels:** Significant reduction in launch overheads and improved throughput justify enabling MPS.
- **Enable MPS for mixed workloads:** Combining compute-bound and memory-bound kernels achieves efficient overlap and better GPU utilization.
- **Cautiously evaluate MPS for saturated compute-heavy workloads:** Measure performance impact carefully; benefits may be limited or negligible.
- **Monitor resource utilization:** Tools like `nvidia-smi` and `cuda-memcheck` can reveal bottlenecks (e.g., SM occupancy, memory bandwidth) that affect MPS effectiveness.
- **Prefer complementary workload mixtures:** Concurrent execution of memory- and compute-bound kernels often yields the best improvements with MPS.

Understanding these nuances allows system architects and application developers to make informed decisions when deploying MPS in multi-tenant, HPC, or inference-serving environments.

7 FUTURE WORK

While this study comprehensively characterizes the impact of MPS across diverse workloads, several avenues for further research remain open:

7.1 Multi-GPU and Cluster Environments

Our experiments were limited to a single-GPU system. In multi-GPU deployments, additional factors such as peer-to-peer communication overhead, NUMA effects, and load balancing arise. Future work should explore MPS behavior in multi-GPU clusters, examining how cross-device communication and scheduling impact concurrency and performance.

7.2 Dynamic Scheduling and Adaptive Resource Management

MPS currently aggregates kernel launches passively. An interesting research direction involves intelligent, dynamic scheduling frameworks that can prioritize kernel launches based on real-time GPU resource availability, memory contention, and application-level QoS requirements.

7.3 Emerging GPU Architectures

Emerging GPU architectures such as NVIDIA Hopper and Ampere introduce new features including asynchronous execution graphs, larger L2 caches, and improved SM partitioning capabilities [7, 9]. Evaluating the interplay between MPS and these hardware advancements presents an exciting avenue for future research. Understanding how modern architectural features affect resource contention, context switching efficiency, and kernel overlap will be crucial in designing next-generation GPU concurrency frameworks.

Additionally, exploration of MPS-aware scheduling in serverless computing frameworks and AI inference pipelines could unlock new opportunities for dynamic, demand-aware GPU sharing, addressing

the growing demand for scalable deep learning and data analytics workloads.

7.4 Energy Efficiency and Thermal Management

Energy efficiency is increasingly important in large-scale deployments. MPS’s impact on GPU power draw, thermal throttling behavior, and long-term system stability should be studied to provide a holistic view of its trade-offs beyond performance metrics alone.

7.5 Expanding Workload Diversity

Although we explored compute- and memory-bound kernels, extending the study to sparse computation, graph analytics, and irregular data access patterns would generalize findings further. Such workloads may exhibit different MPS scaling behaviors worthy of detailed analysis.

8 CONCLUSION

In this paper, we systematically analyzed the performance impact of NVIDIA’s Multi-Process Service (MPS) across a wide spectrum of CUDA workloads. Our study reveals that while MPS consistently improves GPU throughput by reducing kernel launch overheads and increasing concurrency, the extent of improvement is workload-dependent.

Key insights include:

- **Light workloads** benefit greatly when many small kernels are executed concurrently, leveraging MPS’s ability to reduce launch latency.
- **Heavy or mixed workloads** realize substantial gains due to effective overlapping of compute and memory resource demands.
- **Highly saturated heavy workloads** see only marginal improvements, highlighting resource contention as a limiting factor.

By providing experimental evidence, architectural analysis, and practical recommendations, our study equips system designers and application developers with deeper understanding to make informed decisions about MPS deployment.

As GPU sharing continues to evolve in HPC, AI inference, and serverless computing, understanding the nuanced trade-offs of mechanisms like MPS remains crucial. We hope our findings serve as a foundation for further optimizations in GPU concurrency management.

9 ARTIFACTS AND REPRODUCIBILITY

9.1 Artifacts Availability

The full source code, experimental scripts, and result logs are available at:

- <https://github.com/basnetmausam/gpu-mps-sharing-project>

The repository includes:

- Source code (src/) for all CUDA kernels.
- Experiment automation scripts (scripts/).
- Results and profiling outputs (results/).
- Plotting scripts and generated graphs (plots/).
- Instructions for setup and execution.

All experimental code, scripts, and results have been organized to facilitate reproducibility.

9.2 Environment Requirements

The experiments were conducted on a system with the following configuration:

- **GPU:** NVIDIA T1000 (4 GB GDDR6 Memory)
- **CUDA Toolkit:** Version 12.8
- **NVIDIA Driver:** Version 570.124.06
- **Operating System:** Rocky Linux 8.8 (64-bit)
- **Python Libraries:** pynvml, matplotlib, pandas

9.3 Running Experiments

To reproduce the experiments, the following steps should be followed:

- (1) Compile the CUDA programs:


```
cd <project_directory>
make
```
- (2) Run the baseline total experiment (original CUDA workloads):


```
cd scripts
bash run_total_time.sh
```
- (3) Run the mixture workload experiments:


```
bash run_all_mixtures.sh
```
- (4) Generate plots from record logs:


```
python3 plot_graph.py
```

9.4 Notes

- Before running MPS experiments, ensure that no existing MPS daemon is active using `nvidia-smi`.
- Make sure the NVIDIA driver is configured to allow MPS control (EXCLUSIVE_PROCESS mode).

REFERENCES

- [1] Jacob T. Adriaens, Katherine J. Compton, and Wen-Mei W. Hwu. 2012. Flexibility and Scalability in Multi-Core GPU Designs. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 96–106.
- [2] Jianbin Fang, Ana Lucia Varbanescu, and Henk Sips. 2011. A Comprehensive Performance Comparison of CUDA and OpenCL. In *Proceedings of the International Conference on Parallel Processing (ICPP)*. IEEE, 216–225.
- [3] Jianbin Fang, Zili Zhan, Ana Lucia Varbanescu, and Henk Sips. 2020. FaS-GPU: Fine-grained GPU Sharing for Serverless Inference. In *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*. ACM.
- [4] Paras Jain, Xiangyu Wang, and Jianbin Chen. 2018. Efficient GPU Scheduling for Deep Learning Applications. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC)*. IEEE, 1–12.
- [5] Yifan Liu, Jian Xu, Yangdong Fan, Zhiyi Yu, and Xiaoyao Zhou. 2018. Simultaneous Multi-Kernel GPU: Multi-Tasking Throughput Processors via Fine-Grained Sharing. *IEEE Trans. Comput.* 67, 9 (2018), 1205–1219.
- [6] Sayan Mukherjee, Aravind Venkat, Anupam Chattopadhyay, and Alex Orailoglu. 2015. Characterizing Multi-Process Service Execution on GPUs. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 215–224.
- [7] NVIDIA Corporation. 2020. NVIDIA Ampere Architecture Whitepaper. NVIDIA Technical Report. Available at <https://developer.nvidia.com/ampere-architecture>.

- [8] NVIDIA Corporation. 2021. CUDA Multi-Process Service Overview. NVIDIA Developer Documentation. Available at <https://docs.nvidia.com/deploy/mps/index.html>.
- [9] NVIDIA Corporation. 2022. NVIDIA Hopper Architecture Whitepaper. NVIDIA Technical Report. Available at <https://www.nvidia.com/en-us/data-center/hopper/>.
- [10] Yu Wang, Linnan Shi, Kuan Ren, and Wen-Mei W. Hwu. 2023. StreamBox-HBM: Stream Analytics on High Bandwidth Memory Integrated GPU. In *Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC)*. USENIX.
- [11] Qiumin Xu, Udit Wickramasinghe, Jaliya Ekanayake, Yutong Lu, and Bingjing Liu. 2016. Warped-Slicer: Efficient Intra-SM Slicing through Dynamic Resource Partitioning for GPU Multiprogramming. In *Proceedings of the 45th International Conference on Parallel Processing*. IEEE, 1–10.